

ARTI403 – Lab 4

Intensity Transformations and Filtering in the Spatial Domain

Name: Ali Jafar Altaweel

ID: 2240006456

Section: 7MA2

GitHub: <https://github.com/Ali-developer/Image-Processing/tree/main/Lab4>

In this lab, I worked on two families of intensity transformations: thresholding, which turns a grayscale image into a binary one, and histogram processing, which changes the distribution of the gray levels to improve the contrast. The main objective was to understand how the histogram of an image describes its contrast, and how reshaping it changes the image.

Lab Tasks

Task 1 – Contrast stretching

Using the same "moon image", rescale intensity values to include all the intensities that fall within the 3rd and 80th percentiles, and plot the histogram.

Task 2 – Histogram equalization

Using the same "moon image" and the `exposure.equalize_hist` function, display the image and the histogram of the image after flattening the histogram.

Task 3 – Histogram matching

Use the rocket (as reference) and Chelsea images (from `skimage.data`) and implement histogram matching.

Images Used

For the thresholding I used `Parrot.png`, a 453×340 color photograph of a macaw from the Hands-On Image Processing with Python book, which the manual refers to but does not include. I loaded it in grayscale.

For the histogram processing I used three images from `skimage.data`: the moon, a 512×512 grayscale image with very low contrast, the rocket as the reference image, and Chelsea the cat as the source image. All four images are saved in the `images` folder of this lab.

The manual reads the parrot from `../images/`, outside the lab folder. Since the submission requires the images to be inside the lab folder, I changed the path to `./images/` and kept the original line as a comment.

Thresholding

I applied `cv2.threshold()` with the five fixed values from the manual. Instead of opening a separate window for each result, I displayed them all in one figure.

The results showed how sensitive thresholding is to the chosen value:

- At 0, every pixel was white, so the threshold separated nothing
- At 50, still 99% of the image was white

- At 100, about 63% was white and the parrot's dark feathers started to separate from the background
- At 150, only 19% was white, mostly the face of the parrot, the beak and the bright background spots
- At 200, only 3% remained, the brightest highlights

I also plotted the histogram with the thresholds on it and tried Otsu's method, which chose a threshold of 128 automatically from the histogram, between the two thresholds that looked best to me.



Figure 1 – Thresholding the parrot with fixed thresholds

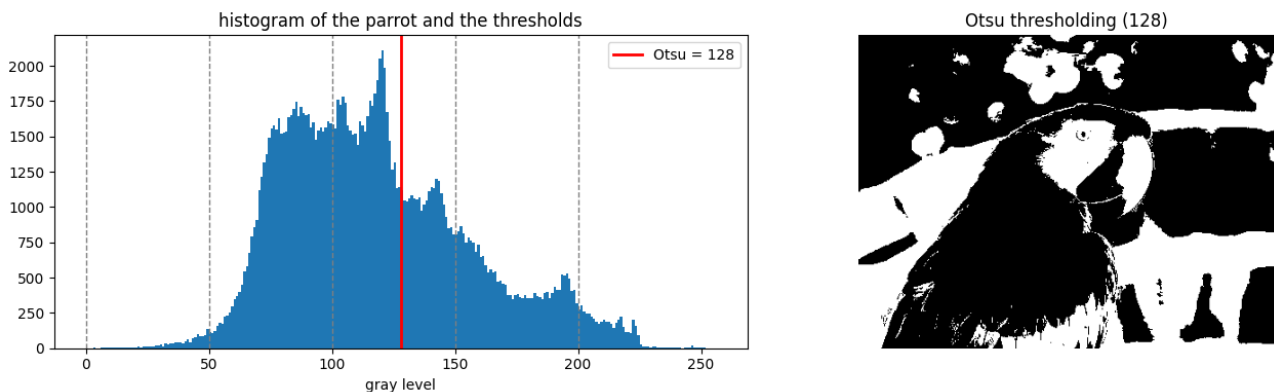


Figure 2 – Histogram with the thresholds, and Otsu's result

Contrast Stretching

The histogram of the moon showed that almost all its pixels were squeezed into a narrow band: the 2nd percentile was 78 and the 98th was 129. This is why the image looks gray and flat.

Using `exposure.rescale_intensity()`, I mapped that range onto the full 0 to 255 range. The histogram spread out over the whole axis and the craters became clearly visible. The 2% darkest and brightest pixels were clipped to 0 and 255, which is the price of this method.

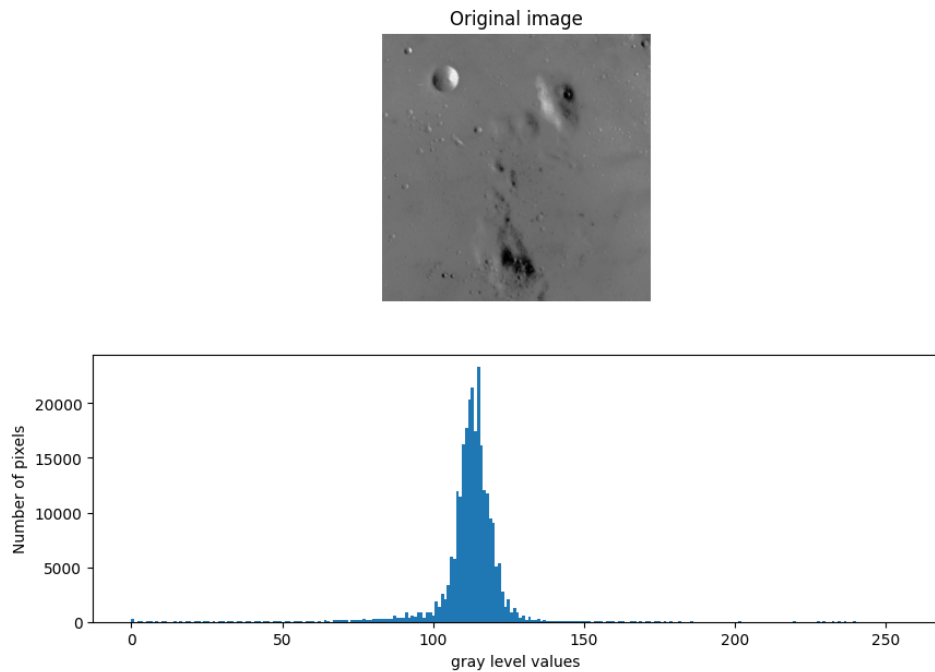


Figure 3 – The original moon image and its histogram

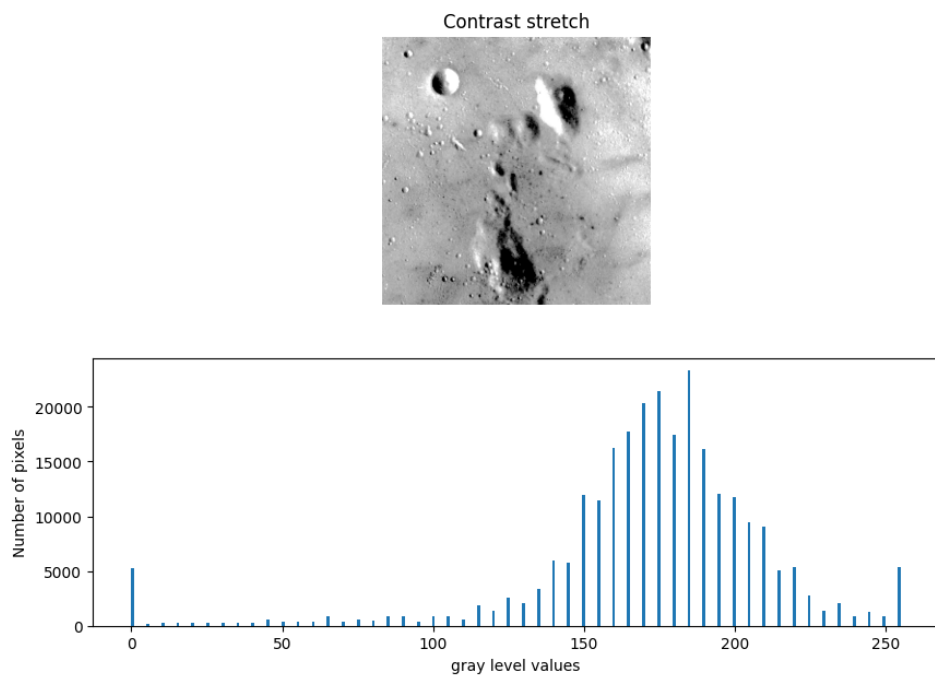


Figure 4 – Contrast stretching between the 2nd and 98th percentiles

Stretching Between the 3rd and 80th Percentiles

For the first task I repeated the stretching with the 3rd and 80th percentiles, which were 87 and 118. Because this range is narrower, the mid tones gained even more contrast than with the 2 to 98 stretch.

However, the 80th percentile is a strong cut. In total 16.5% of the pixels were above it and all became 255, which showed up as a tall spike at the right end of the histogram and as burned out white areas on the surface of the moon. Only 2.8% were clipped at the dark end.

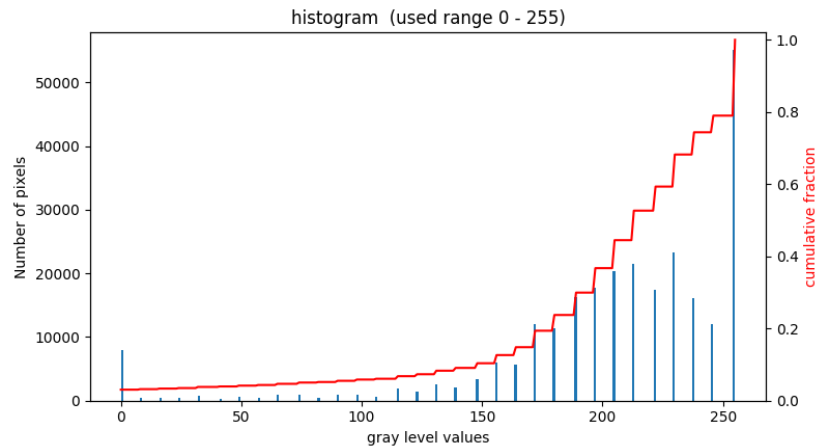
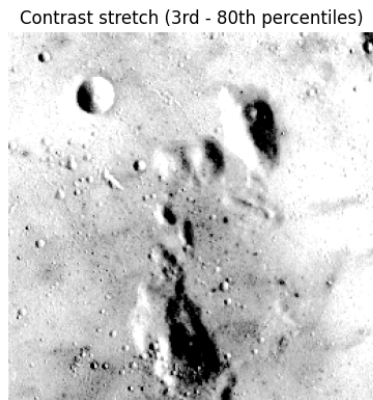


Figure 5 – Task 1: stretch between the 3rd and 80th percentiles with its histogram

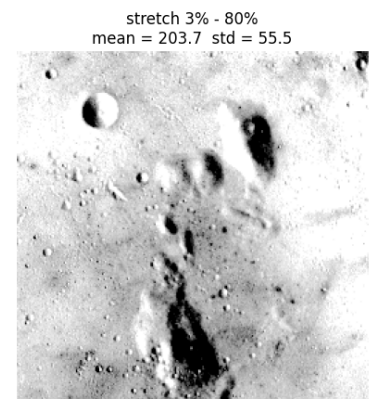
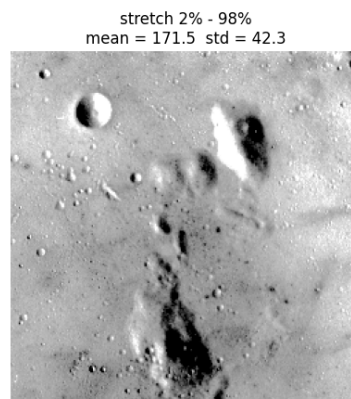
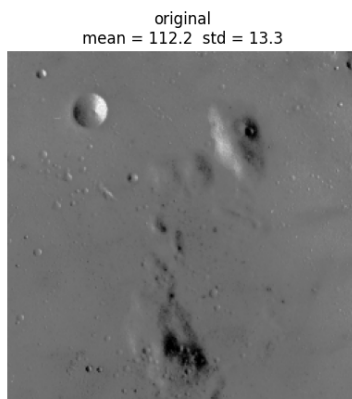


Figure 6 – Task 1: comparison of the two stretches

Histogram Equalization

For the second task I used `exposure.equalize_hist()`. It returns floating point values between 0 and 1, not 0 to 255, so the manual's histogram code with a range of 0 to 255 would have shown everything in the first bin. I converted the result back to 8-bit with `img_as_ubyte()` before plotting.

The standard deviation went from 13.3 to 73.9, so the contrast increased more than five times. The cumulative histogram of the equalized image became almost a straight line, never more than 0.085 away from the ideal uniform line.

The histogram itself was not flat but made of separate bars with gaps between them. This is because all the pixels sharing one gray level are mapped together, so equalization can move the bars apart but it can never split one.

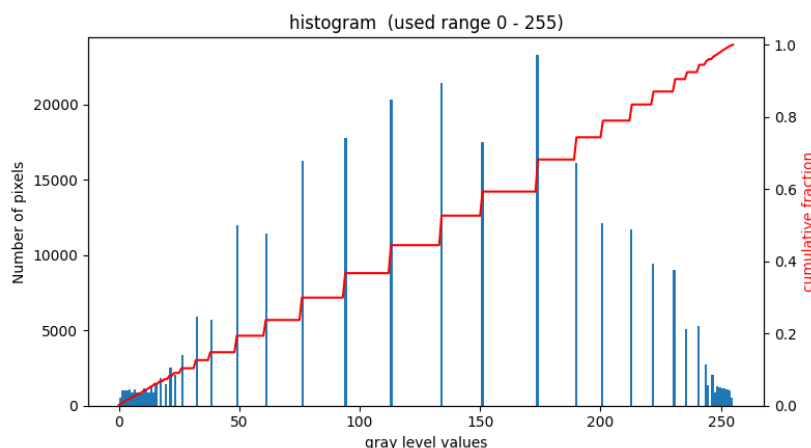


Figure 7 – Task 2: the equalized moon image and its histogram

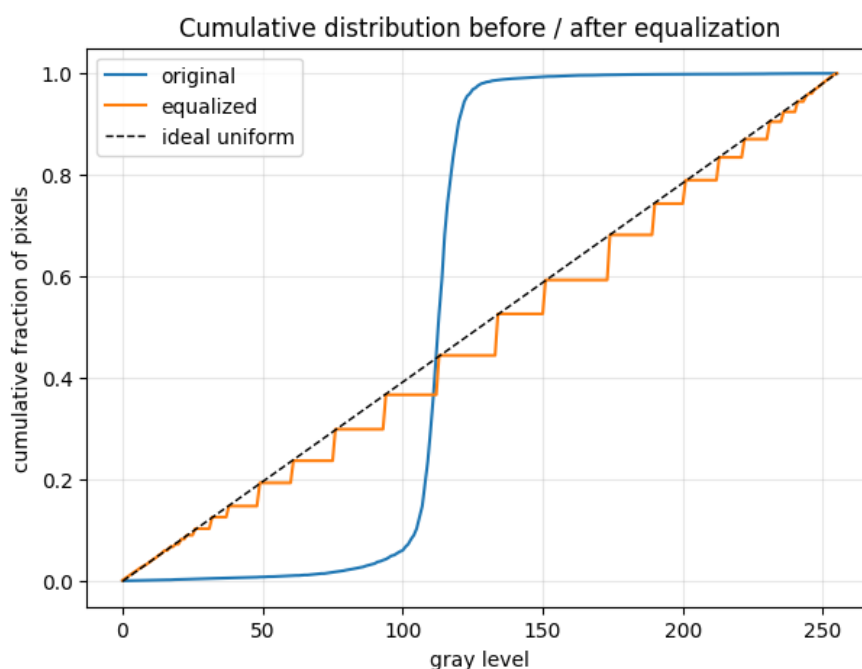


Figure 8 – Task 2: cumulative distribution before and after

Histogram Matching

For the third task I used `match_histograms()` with Chelsea as the source and the rocket as the reference, matching each color channel separately. The matched cat kept all its shapes and edges, but took the dark, bluish colors of the rocket image.

The numbers confirmed it. The mean of the red channel went from 147.7 in the source to 52.1 after matching, almost the 52.3 of the reference, and the green and blue channels followed in the same way. The cumulative histograms of the matched image and of the reference differed by less than 0.02 on every channel. The method works even though the two images have different sizes, because it only uses their histograms.

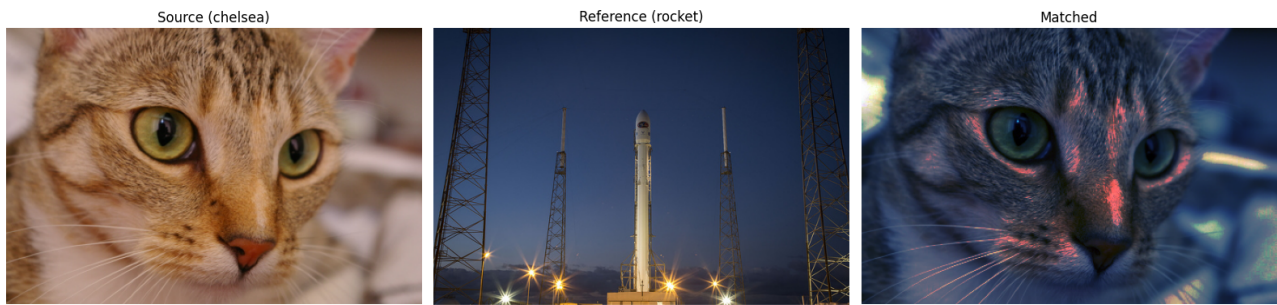


Figure 9 – Task 3: source, reference and matched images

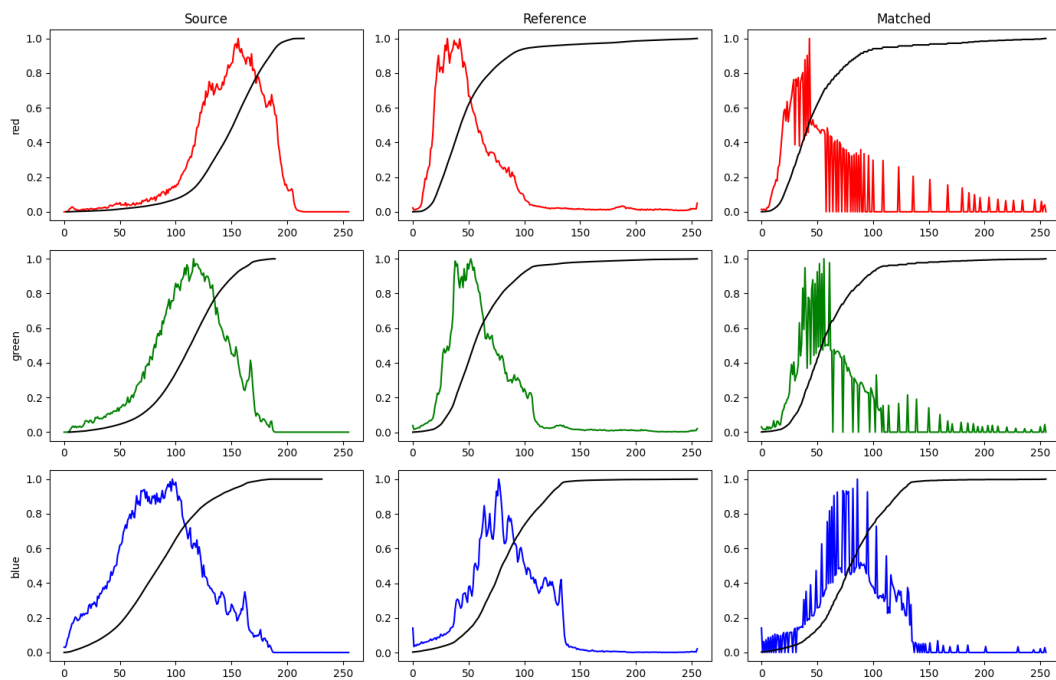


Figure 10 – Task 3: per-channel histograms (color) and CDFs (black)

Observations

The most surprising thing was how much a histogram tells about an image. Just by looking at the moon's histogram, squeezed between 78 and 129, it was clear why the image looked flat, and every method in this lab was simply a different way of reshaping that histogram.

I also noticed that each method has a cost. Fixed thresholding depends completely on the chosen value, contrast stretching clips the extreme pixels, and equalization leaves gaps in the histogram and can over enhance noise in flat regions.

Conclusion

This lab showed that thresholding and histogram processing are both point operations, where each output pixel depends only on the value of the same input pixel. Thresholding reduces the image to two levels to separate objects, while histogram processing spreads the gray levels to improve contrast.

Contrast stretching gives control over which part of the range is enhanced, histogram equalization works automatically and increased the moon's contrast more than five times, and histogram matching transfers the tone of one image onto another.

Submission

The complete work for this lab is in my GitHub repository, in the **Lab4** folder:
<https://github.com/Ali-developer/Image-Processing/tree/main/Lab4>

As required, the Lab4 folder contains the following files:

- Jupyter notebook: Lab4.ipynb, executed, with all the outputs and the task solutions
- Images used: images/Parrot.png, images/chelsea.png, images/moon.png, images/rocket.png
- This report: Lab4_Report.pdf, and the same write-up as README.md